Phase Finale du Projet (Jour 5–6)

Contraintes avancées et extensions du modèle

Projet : Optimisation logistique numérique

Objectif

Durant les deux derniers jours du projet, de nouvelles contraintes sont introduites afin de simuler l'évolution d'un projet réel.

Les étudiants doivent adapter leur modèle mathématique, modifier leur implémentation Python et mettre à jour leur interface utilisateur.

Chaque groupe doit implémenter **au minimum trois extensions** parmi celles proposées ci-dessous.

1 Contraintes d'optimisation avancées

1.1 Activation binaire des centres

Un centre de traitement ne peut être activé que s'il traite un nombre minimal de requêtes.

On introduit une variable binaire :

$$y_j \in \{0, 1\}$$

où :

— $y_j = 1$ signifie que le centre C_j est activé

— $y_j = 0$ signifie qu'il est désactivé

Contraintes :

$$\sum_i x_{ij} \leq Cap_j \, y_j$$

$$\sum_i x_{ij} \geq 200 \, y_j$$

Les étudiants doivent transformer leur problème en **programmation linéaire entière**.

1.2 Coût fixe d'activation

Chaque centre utilisé entraîne un coût fixe.

Centre	Coût fixe
C_1	1000
C_2	800
C_3	600

La fonction objectif devient :

$$\min \sum_{i,j} c_{ij} x_{ij} + \sum_j F_j y_j$$

2 Contraintes énergétiques

Chaque centre consomme de l'énergie par requête traitée.

Centre	Consommation énergétique
C_1	2 unités
C_2	1.5 unités
C_3	2.5 unités

Une limite globale est imposée :

$$\sum_{i,j} e_j x_{ij} \leq E_{max}$$

Les étudiants doivent analyser l'impact de cette contrainte sur la solution optimale.

3 Contraintes réseau

3.1 Latence maximale

La latence entre chaque région et chaque centre est donnée par :

	C_1	C_2	C_3
R_1	20	30	40
R_2	25	15	35
R_3	30	20	25
R_4	50	30	15

Une contrainte est imposée :

La latence maximale acceptable est de 35 ms.

Les flux qui dépassent cette limite doivent être interdits :

$$x_{ij} = 0$$

3.2 Distance réseau

On ajoute un coût proportionnel à la distance réseau :

$$\min \sum_{i,j} (c_{ij} + \alpha d_{ij}) x_{ij}$$

où d_{ij} représente la distance réseau.

4 Contraintes de fiabilité

4.1 Redondance obligatoire

Chaque région doit être traitée par au moins deux centres différents.
Les étudiants doivent introduire une contrainte garantissant la redondance des flux.

4.2 Simulation de panne

Une panne peut survenir sur un centre.
Exemple :

Le centre C_2 devient indisponible.

Les étudiants doivent :

- recalculer la solution optimale
- modifier l'IHM pour permettre la simulation d'une panne

5 Données dynamiques

Les requêtes peuvent évoluer dans le temps.
On introduit une variable :

$$D_{i,t}$$

qui représente la demande de la région i à la période t.
Les étudiants doivent résoudre le problème pour plusieurs périodes.

6 Contraintes de développement logiciel

6.1 Sauvegarde des scénarios

L'application doit permettre :

- sauvegarde des scénarios
- chargement des scénarios

Le format de sauvegarde recommandé est :

`JSON`

6.2 Mode simulation

L'interface utilisateur doit permettre :

- tester plusieurs scénarios
- comparer les résultats
- visualiser les différences

6.3 Export des résultats

L'application doit permettre d'exporter les résultats sous forme :

- CSV
- PDF

7 Contraintes de performance

Le problème doit être étendu à :

- 8 régions
- 6 centres

Les étudiants doivent analyser :

- le temps de calcul
- la robustesse de leur implémentation

8 Optimisation multi-objectifs

On introduit une optimisation multi-critères :

- coût
- latence
- énergie

La fonction objectif devient :

$$\min \alpha Cost + \beta Latence + \gamma Energie$$

Les étudiants doivent tester plusieurs combinaisons de poids.

Organisation recommandée

Jour	Activité
1	Modélisation mathématique
2	Programmation linéaire
3	Implémentation Python
4	Interface utilisateur
5	Contraintes avancées